Assignment 3: Save the Campus Festival

Version 1.0.1 | English student specification

The campus festival opens soon. The waffle tent has power, the stage has a plan, and the emergency tea station has neither. Your console toolkit will connect venues, find circular prerequisites, schedule preparations and route supplies. Each graph has its own purpose: undirected cables, directed dependencies, and directed acyclic delivery routes.

Learning goals

Implement, explain and validate DSU, weighted graphs, DFS, SCC, topological order and DAG relaxation. Recognise ties and edge cases; reconstruct valid answers and justify complexity.

Prerequisites

Adjacency lists, DFS, priority queues, sorting and 64-bit arithmetic.

Work in small milestones

Connectivity -> cable forest -> dependency groups -> schedule -> delivery routes. These are three different graph models; do not reuse undirected cable edges as directed dependencies.

Practical rubric (50 points)

Task	Algorithm	Requirement group	Points
1	dsu	core	4
1	dsu	edge	2
1	dsu	scale	2
2	kruskal	core	4
2	prim	core	4
2	kruskal	witness	2
2	prim	witness	2
2	kruskal	scale	1
2	prim	scale	1
3	scc	core	5
3	scc	witness	3
3	scc	scale	2
4	topo	core	4
4	topo	edge	2
4	topo	scale	2
5	dag	core	5
5	dag	witness	3
5	dag	scale	2

core: optimum values/basic behaviour; edge: published edge cases; witness: feasible optimal reconstruction/condensation; scale: correct large-instance results. Each group has a fixed maximum. The sum is 50.

Task 1 - Connectivity / DSU

Connect the waffle tent, quiet stage and emergency tea station, one cable at a time.

Input: {task:dsu,n,operations:[[u,v],...]}

Output: {states:[[components,largest],...]}

Limits: 0 <= n <= 100000; 0 <= operations <= 250000. If n=0, operations must be empty. Self-union and repeated union are valid.

Implement find and union, path compression and union by size or rank. Initially n singleton components; largest=0 for n=0. Return a state after every operation. A union within one component does not change the count or size.

Complexity: O((n+q) alpha(n)) amortized time, O(n+q) memory including output.

Example input:

{"task": "dsu", "n": 4, "operations": [[0, 1], [1, 2], [0, 2], [3, 3]]}

One accepted output:

{"states": [[3, 2], [2, 3], [2, 3], [2, 3]]}

The third cable is redundant. The tea station remains alone; connecting it to itself does not help.

Task 2 - Budget Network / Kruskal

Build the cheapest undirected cable network; finance would prefer cables to an inspirational speech.

Input: {task:kruskal,n,edges:[[u,v,weight],...]}

Output: {cost,edge_ids:[...],components}

Limits: 0 <= n <= 100000; 0 <= m <= 250000; -1000000000 <= weight <= 1000000000. Parallel edges and self-loops allowed.

Implement Kruskal using DSU. Return a minimum spanning forest, including all connected components and isolated vertices; components=0 for n=0. cost is the total forest cost. Return input edge IDs, with no repetitions. Self-loops are excluded. Any optimal forest is accepted. Negative weights represent discounts.

Complexity: O(n+m log(m+1)) time, O(n+m) memory.

Example input:

{"task": "kruskal", "n": 4, "edges": [[0, 1, 4], [1, 2, 2], [0, 2, 3]]}

One accepted output:

{"cost": 5, "edge_ids": [1, 2], "components": 2}

Venue 3 is isolated. The best available forest costs 5; it does not connect every venue.

Task 2 - Budget Network / Prim

Independently build the same minimum-cost forest by expanding one component at a time.

Input: {task:prim,n,edges:[[u,v,weight],...]}

Output: {cost,edge_ids:[...],components}

Limits: Same as Kruskal.

Implement Prim with adjacency lists and a priority queue. Restart at every unvisited component. Use the same forest contract as Kruskal. Costs and feasibility must agree, but selected edge IDs may differ. Compare algorithms on three input sizes in ANALYSIS.md.

Complexity: O(n+m log(m+1)) for a lazy heap; O((n+m) log(n+1)) for an indexed heap; O(n+m) memory.

Example input:

{"task": "prim", "n": 3, "edges": [[0, 1, 1], [1, 2, 1], [0, 2, 1]]}

One accepted output:

{"cost": 2, "edge_ids": [0, 2], "components": 1}

Several answers are optimal. The validator checks the forest rather than matching one reference edge list.

Task 3 - Circular Dependencies / SCC

Find groups such as 'print tickets before opening the printer room' and 'open the printer room with a printed ticket'.

Input: {task:scc,n,edges:[[u,v],...]}

Output: {component:[label per vertex],condensation:[[from_label,to_label],...]}

Limits: 0 <= n <= 100000; 0 <= m <= 250000. Directed edges; duplicates and self-loops allowed.

Implement Kosaraju with two DFS passes and a transposed graph. Labels are nonnegative integers; their values and order are unrestricted. Equal labels must mean exactly one SCC. Condensation contains each cross-component pair once and no internal edges. Handle isolated vertices and deep graphs. Condensation is a DAG, but internal dependency cycles still need a real scheduling decision.

Complexity: O(n+m) expected time with a hash set for edge deduplication; O(n+m) memory.

Example input:

{"task": "scc", "n": 4, "edges": [[0, 1], [1, 0], [1, 2], [0, 2], [2, 3]]}

One accepted output:

{"component": [5, 5, 7, 9], "condensation": [[5, 7], [7, 9]]}

Vertices 0 and 1 form one SCC. Two original edges become one condensation edge.

Task 4 - Festival Schedule / Topological Ordering

Schedule preparations before the band arrives to discover that the stage is still a diagram.

Input: {task:topo,n,edges:[[u,v],...]}

Output: {acyclic:boolean,order:[...]}

Limits: Same directed graph domain as SCC. Edge u->v means u must finish before v starts.

Implement DFS with unvisited/active/finished states and reverse postorder. A back edge to an active vertex indicates a cycle. For a cyclic graph return acyclic=false, order=[]. For a DAG list every vertex exactly once, respecting every edge. Any valid order is accepted. The empty graph has acyclic=true, order=[].

Complexity: O(n+m) time and memory, including explicit DFS stack.

Example input:

{"task": "topo", "n": 4, "edges": [[0, 2], [1, 2], [2, 3]]}

One accepted output:

{"acyclic": true, "order": [1, 0, 2, 3]}

Preparations 0 and 1 may be swapped; both must precede 2.

Task 5 - Delivery Routes / DAG Shortest Paths

Route supplies through a directed acyclic delivery graph. A downhill conveyor can give a negative edge cost.

Input: {task:dag,n,edges:[[u,v,weight],...],source}

Output: {acyclic:boolean,distance:[integer or null per vertex],parent_edge:[edge ID or -1 per vertex]}

Limits: 1 <= n <= 100000; 0 <= m <= 250000; -1000000000 <= weight <= 1000000000; 0 <= source < n. Directed graph; parallel edges and self-loops allowed. Cyclic input must be rejected as below.

Check acyclicity, then relax edges in topological order. On ANY directed cycle, including an unreachable cycle, return acyclic=false and both arrays empty. Otherwise source has distance=0 and parent_edge=-1. Unreachable vertices have distance=null and parent_edge=-1. Every other parent edge ends at that vertex and reconstructs an optimal source path. Provide a route reconstruction helper and show it in an example. Never relax from infinity. Multiple shortest routes are accepted; MST is unrelated to this objective.

Complexity: O(n+m) time and memory. Route reconstruction costs O(route length).

Example input:

{"task": "dag", "n": 5, "source": 0, "edges": [[0, 1, 3], [0, 2, 8], [1, 2, -7], [2, 3, 4]]}

One accepted output:

{"acyclic": true, "distance": [0, 3, -4, 0, null], "parent_edge": [-1, 0, 2, 3, -1]}

Route to 3 is 0->1->2->3 with cost 0. Venue 4 is unreachable. Negative DAG weights are valid.

Common transport and reset rules

One JSON object per input line, one JSON result object per line. One task per request. No prompts or debug output on stdout. Independent requests reset all task state. Provided adapters handle input/output. Inputs satisfy the published domain; malformed JSON is not graded.

UTF-8 JSON lines. Task names are case-sensitive.

All vertices and edges use zero-based indices. An edge ID is its position in the input edge array. All costs, distances, weights and counts use signed 64-bit integers; null denotes unreachable distance.

Assessment: exactly 50 + 50

Each assignment has practical work worth 50 and an individual in-class quiz worth 50. Their sum is the assignment score out of 100. Each assignment contributes 20% of the second attestation; the separate Quiz II and Endterm keep their syllabus weights of 20% and 40%. These new quizzes are part of the assignments, not replacements for Quiz II.

Practical points belong to published requirement groups. Within each group: group points * passed checks / total checks, rounded to 0.01 with half-up rounding. The practical sum is capped at 50. Correct optimum values can earn core points even when a witness is invalid; reconstruction points require a valid optimal witness. Scale groups require correct output on published large input sizes within generous resource limits. A failed build prevents checks from earning points. A timeout loses the affected group checks. An internal checker error produces no grade and must be investigated and rerun.

Official checks use the submitted commit and a fixed version of the instructor checker, not your repository's workflow. Additional tests do not change a group's maximum points. Test data may be private, but all tested requirements are published here. Timing medians are for analysis; you are not ranked by milliseconds. The checker records functional correctness and scalability. Black-box tests alone cannot prove which algorithm you used; the stated implementation methods and individual understanding remain required.

Individual understanding quiz

The individual quiz is a separate part worth up to 50 points. Its question bank and final arrangements are deferred and will be announced before the activity opens. Question count, duration, attempts, question formats and category balance will be confirmed by the instructor. The graded quiz remains hidden during preparation. Practice quizzes A and B are optional preparation and do not affect your grade.

Reconsidering specific quiz errors

You may ask to reconsider points lost on a particular question. This is optional, not a new compulsory defence. The instructor identifies the skill, gives a new equivalent example and asks you to explain the reasoning. Fully demonstrated understanding restores the points lost on that question; a correct main method with one significant unfinished part restores half of the lost points; otherwise the original score remains. Repeating a published answer without explanation is insufficient. The quiz maximum remains 50. Quiz reconsideration does not automatically change practical points. An erroneous question/key is corrected for all affected students.

Copy, run and submit

Open the official template link in Moodle and select Use this template -> Create a new repository. Choose any repository name. Public or private copies are allowed; if private, provide the instructor with access, or use the authorised local checking route. No payment or private repository is required. Do not fork an instructor checker or copy its private content.

Install your chosen runtime: Java JDK 17+, Python 3.12+, C++17 compiler, or .NET SDK 8. Python 3.12+ is needed for the launcher in every language. Set language in submission.json, then run python tools/run.py --setup. After implementing the algorithms, run python tools/run.py --check. Run a UTF-8 request file with python tools/run.py --input examples/dsu.jsonl. All commands use submission.json by default; --language Java/Python/Cpp/CSharp overrides it for that run. Windows may use py -3 in place of python. Setup checks transport only; TODO algorithms intentionally fail --check. See TESTING.md for public cases and reset checks.

Implement the method bodies in your selected language's source folder. Keep the provided JSON transport separate. Add helper files at the top level of that source folder: .java, .py, .cpp/.hpp/.h or .cs, depending on your language. C++ builds all top-level .cpp files; headers are included normally. Nested source folders are not part of the build contract. Standard collections, sorting and priority queues are allowed; external algorithm libraries cannot replace required implementations. C++ targets Linux GCC for official checks. Keep the required methods visible in your source and explain them in ANALYSIS.md so the instructor can inspect that you used DSU with both heuristics, distinct Kruskal/Prim implementations, Kosaraju, DFS topological ordering and DAG relaxation. The public launcher does not prove algorithm choice and introduces no extra scoring group or penalty.

Complete ANALYSIS.md with 3-5 sentences per algorithm, three reproducible comparison input sizes, runtime/seed/commands and observations. Explain your states/invariants and reconstruction. This short report supports learning; it has no separate hidden marking group. Record AI tools and other resources used, their contribution, and what you verified. If none, say so. You must understand your submitted implementation; the quiz checks individual understanding. This project does not claim to prevent AI use.

Commit the finished source, your own tests, ANALYSIS.md and SUBMISSION.md. Set the selected language in submission.json. In Moodle submit your repository URL, full 40-character commit SHA and selected language. Record the SHA in the Moodle submission rather than making a self-referential commit. The official grade refers to that fixed commit, not a changing branch. Do not submit build binaries, credentials, private grader files or quiz keys. Dates and submission availability are provided in Moodle, not invented in this document.

Resource limits

Official compilation: 60 seconds. Each requirement-group batch: 20 seconds, one CPU, 768 MiB container memory (Java heap up to 512 MiB), 128 processes, at most 8 MiB UTF-8 output. Representative large checks use up to 50000 graph vertices, 100000 DSU operations and 60000 MST edges, inside the full input domains in CONTRACTS.md. Limits apply with headroom; output correctness and asymptotic analysis matter, not tiny timing differences on shared CI.